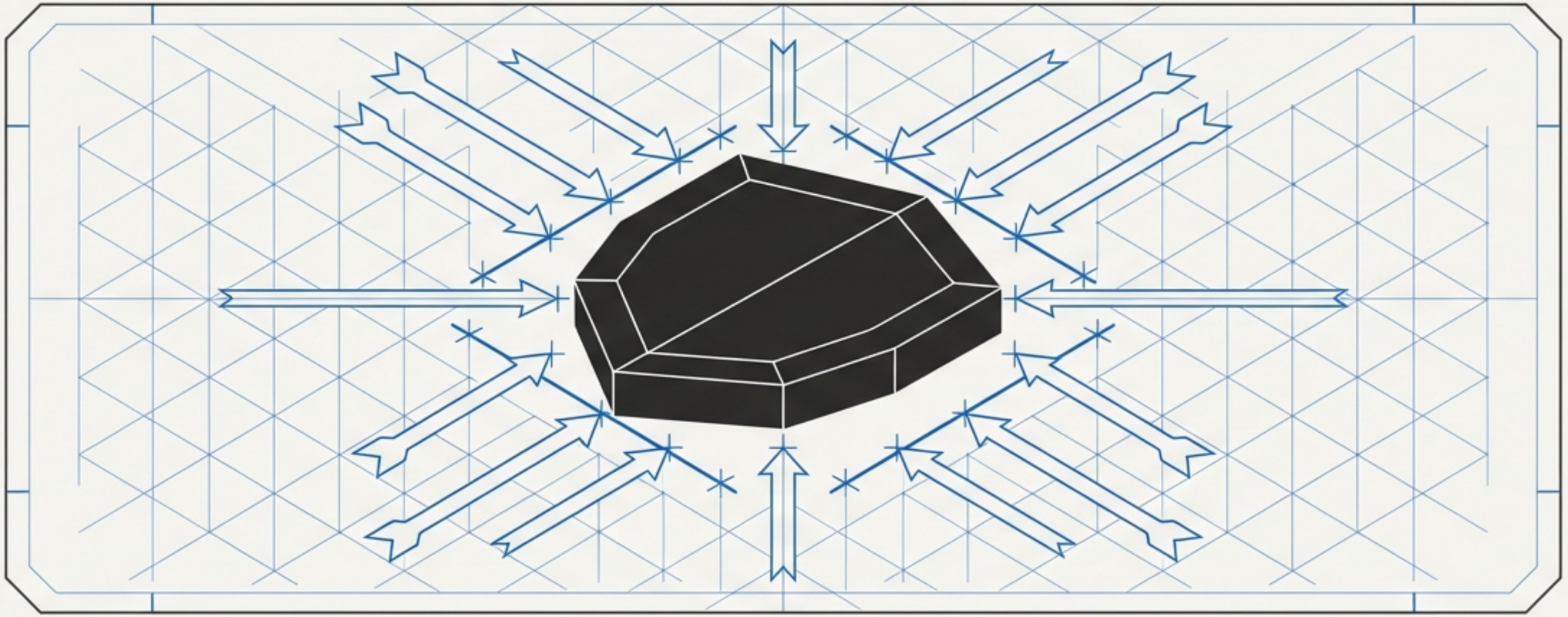
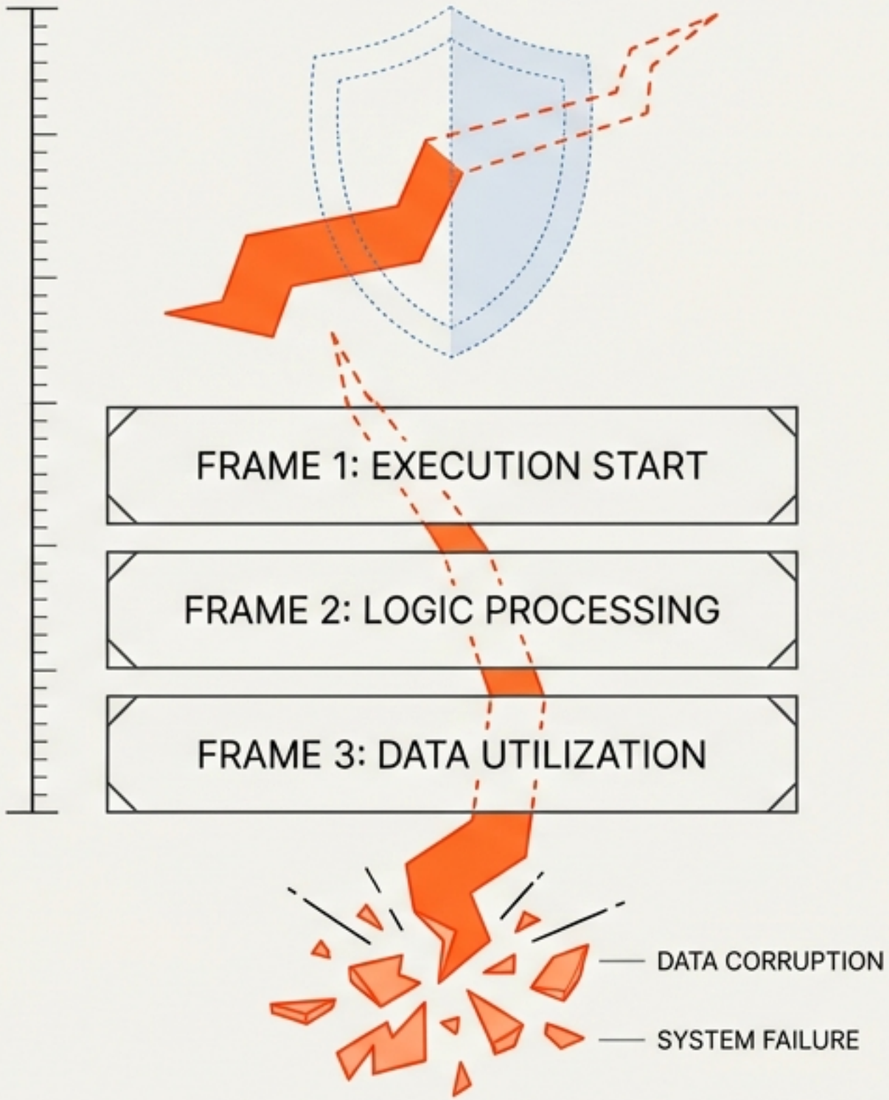
Type_Safe Framework

Comprehensive Reference Guide & Runtime Enforcement Protocol

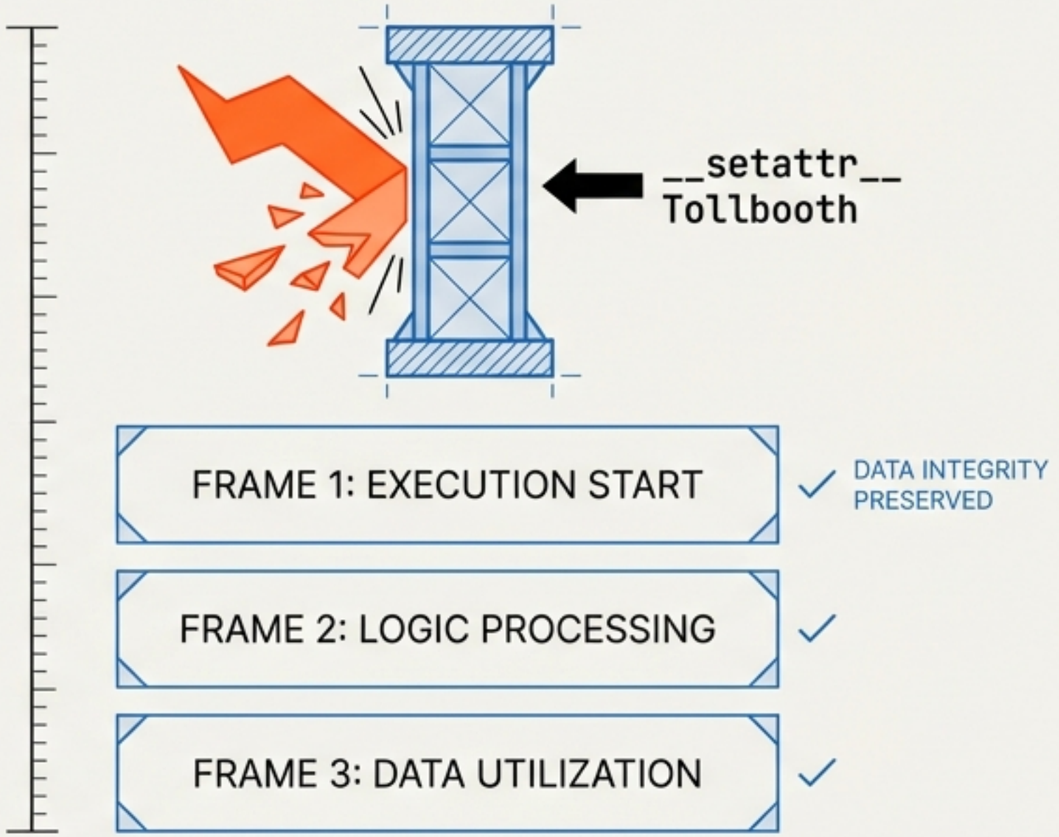


osbot-utils v3.63.4 | Project Context: SG/Send

The Illusion: Documentation Only

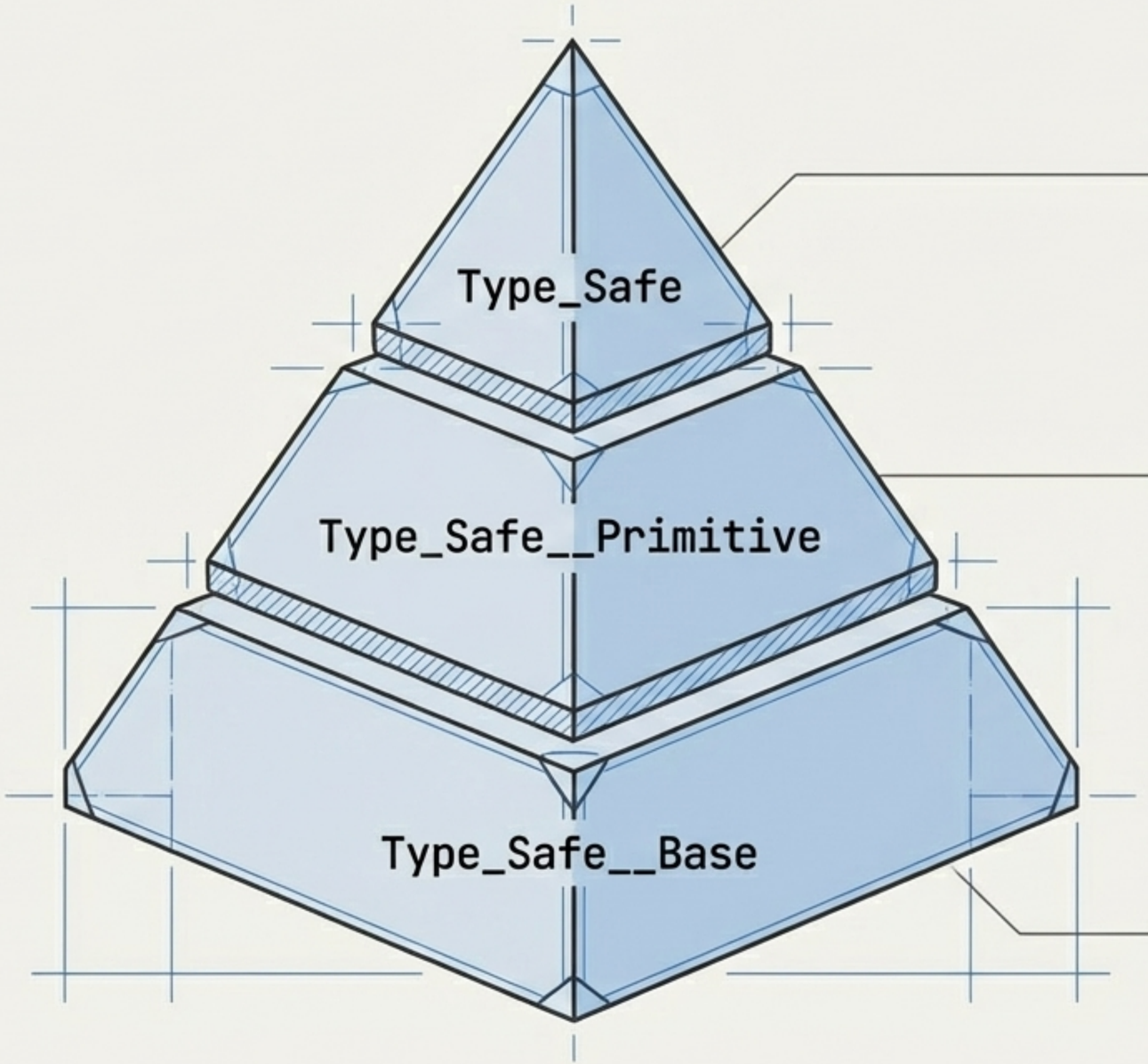


The Reality: Runtime Enforcement



Python **type hints** are ignored at **runtime**. **Type_Safe** catches silent data corruption at the exact point of assignment, not three stack frames later.

The Three-Class Architecture



The user-facing class. Inherited by all operational code. Auto-initialises attributes, enforces types via __setattr__, and manages perfect JSON round-trips.

Extends built-in types (str, int, float), preserves strict types through arithmetic, and provides `__to_primitive__()` for serialisation.

Houses core type checking logic (`try_convert`), and handles complex typings like `Union`, `Literal`, and `Optional`.

The Core Operational Mandates

1. Always inherit from Type_Safe.

2. Type-annotate every attribute.

3. No mutable defaults — ever.

4. BAN RAW PRIMITIVES.

5. Schemas are pure data (no methods).

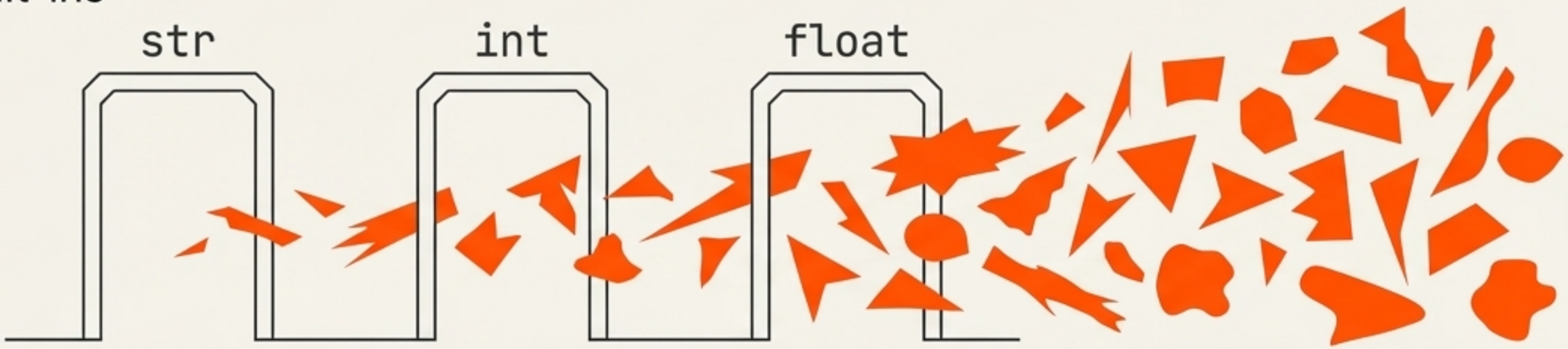
6. Collections are pure type definitions.

7. Never shadow reserved built-in methods.

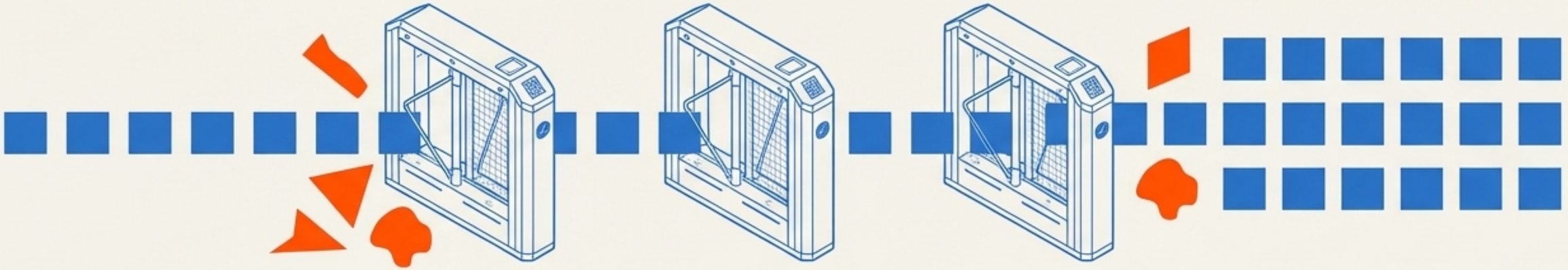
8. Forward references restricted to current class.

The Ban on Raw Primitives

The Threat: Raw built-ins



The Defence: Type_Safe Primitives



Never use raw str, int, or float in a Type_Safe class. They are black holes in the enforcement chain. They accept any data without validation, immediately breaking the strict guarantees the framework provides.

Periodic Table of Safe Primitives

Core Strings

Safe_Str
JetBrains Mono

512 chars, regex-filtered

Safe_Str__Text
JetBrains Mono

4KB limit

Safe_Str__Id
JetBrains Mono

128 chars, alphanumeric + _ -

Domain Strings

Safe_Str__Email
JetBrains Mono

Safe_Str__Url
JetBrains Mono

Safe_Str__Http__Content_Type
JetBrains Mono

[Allows /]

Safe_Str__LLM__Prompt
JetBrains Mono

[Max 32KB]

Safe_Str__Slug
JetBrains Mono

[Auto-lowercases]

Numerics

Safe_Int
JetBrains Mono

Safe_UInt__Port
JetBrains Mono

[0-65535]

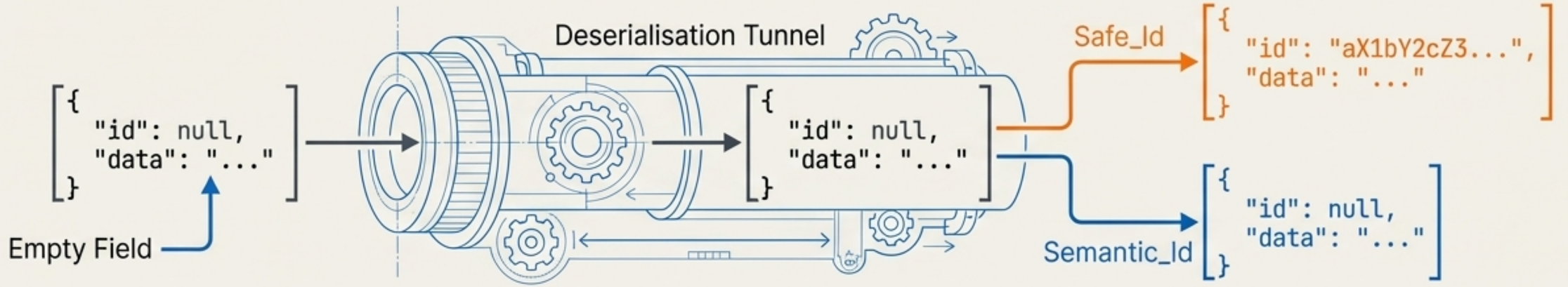
Safe_Float__Money
JetBrains Mono

[Decimal Maths]

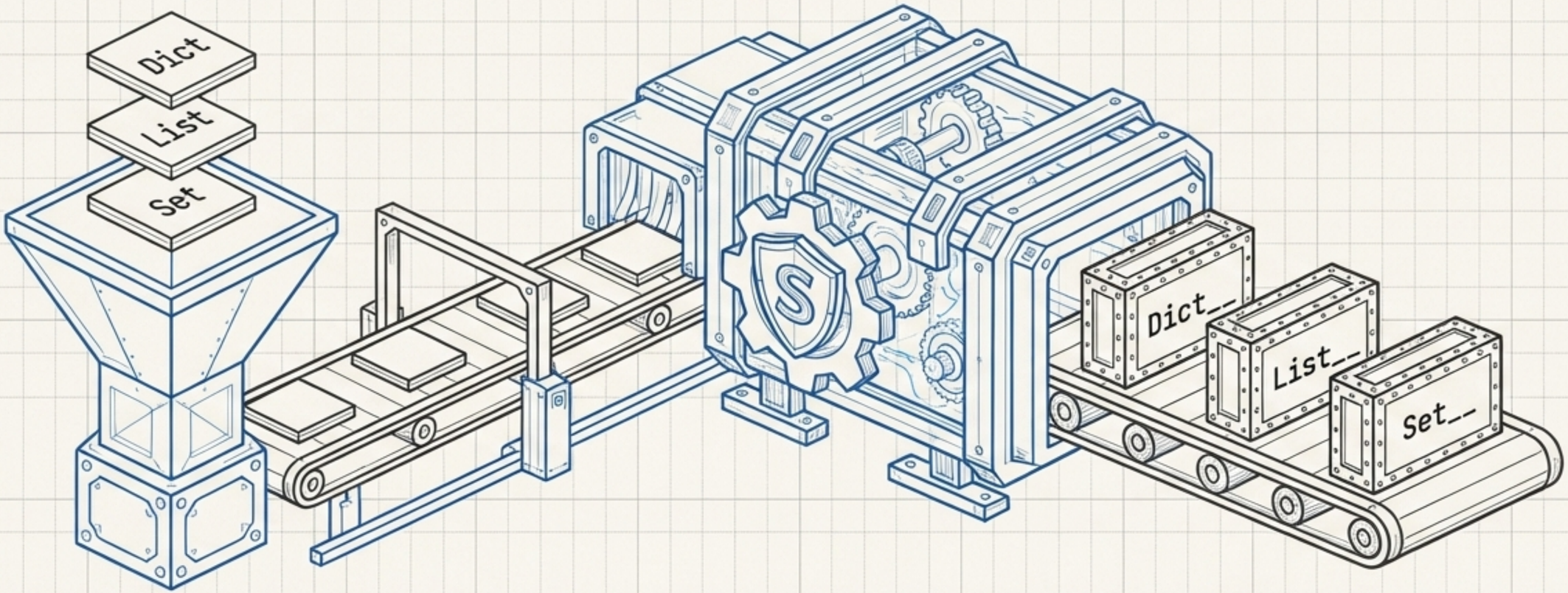
CRUCIAL WARNING: Always test the Safe type with real data. Choosing the wrong type guarantees silent data corruption.

The Identity Crisis: Instance vs. Concept

Safe_Id	Semantic_Id
• **Behaviour**: Auto-generates a random value when None or empty. • **The Risk:** Breaks serialisation round-trips if misused (deserialising an empty value creates a NEW ID rather than restoring the original). • **Use Case:** Instance Identifiers (e.g., Node_Id, Edge_Id, Graph_Id).	• **Behaviour:** Never auto-generates. Explicitly allows empty values. • **The Risk:** None during serialisation. • **Use Case:** Concept Identifiers (e.g., Ontology_Id, Category_Id, Node_Type_Id).

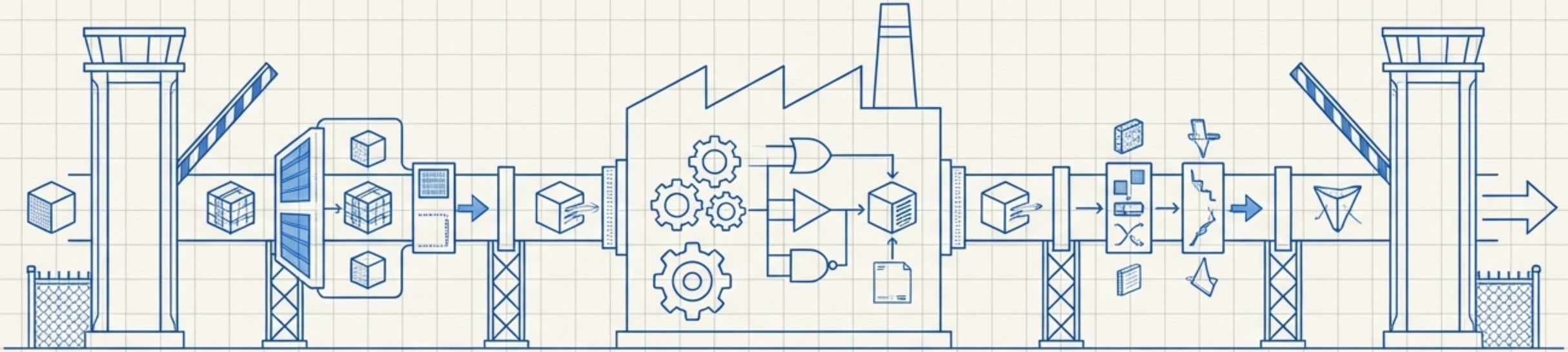


Collection Subclassing Engine



Automatic Interception	The Naming Convention	The Zero-Method Mandate
When a Type_Safe class is constructed, standard annotations are automatically intercepted and converted into strict subclasses.	Always prefix with the collection type. Operations on these collections strictly preserve the subclass type during runtime.	Subclasses must be pure type definitions (expected_type, expected_key_type). Zero methods are permitted.

Two-Phase Validation Tollbooth: @type_safe



Gate 1: Parameter Validation

Internal: Core Execution Logic

Gate 2: Return Value Auto-Conversion

Rules of Engagement

Use `= None` for optional parameters. Do NOT use `Optional[T]`.

Returning `None` is always permitted regardless of annotation.

The Mechanism

Validates parameters strictly on entry.

Auto-converts and mathematically validates return values on exit.

Performance Reality

Applied primarily to public API methods.

The decorator intelligently detects no-parameter methods to skip validation, reducing overhead by ~5x.

The Serialisation Round-Trip

Testing & Inspection Tool	CRITICAL DIRECTIVE
`.obj()` converts the strict schema into a flexible namespace object for rapid, frictionless state verification during unit testing.	⚠ Never shadow reserved method names (e.g., `reset`, `print`, `update_from_kwargs`). Doing so silently destroys the serialisation engine.

Synthesis: Anatomy of a Perfect Type_Safe Service

```
class User_Schema(Type_Safe):
    id: int
    username: str
    email: EmailStr
    created_at: datetime = field(default_factory=datetime.utcnow)
```

The Schema: Pure data. Strict typings. No mutable defaults. Zero logic.

```
class Dict__Users(Dict):
    User_id: User_Schema
```

The Collection: Pure type definition. Enforces key/value constraints. Zero methods.

```
class User_Service(Type_Safe):
    @type_safe
    def get_user(self, user_id: int) -> User_Schema:
        ...
    def create_user(self, data: CreateUserRequest) -> User_Schema:
        ...
```

The Service: Business logic. Inherits Type_Safe. Two-phase validation applied exclusively to public APIs.

Precision Testing Patterns

Context Mastery

Use the _ context manager pattern for clean, isolated scoped testing.

State Verification

Deploy `.obj()` for rapid, comprehensive assertions of object state.

Dynamic Bypasses

Utilise `__SKIP__` to gracefully handle auto-generated, unpredictable values (IDs, timestamps) during assertions.

Performance Overhead

Push expensive resource initialisation to `setUpClass`. This mitigates initialisation overhead, yielding 10-100x speedups.

Reality Check: The SG/Send Audit

Audit Context: SG/Send Framework Review (v0.2.41)

12

Core Files Analyzed

90+

Critical Issues Discovered

Verdict: The rules of Type_Safe are not stylistic preferences. Breaking them leads directly to silent system degradation, API contract violations, and corrupted databases.

Top Silent Killers: Forensic Lessons

1. Silent Corruption

Choosing the wrong `Safe_Str`. Using `Safe_Str__Id` for MIME types silently strips the forward slash, permanently corrupting the data payload.

2. API Contract Leaks

Swagger exposes defaults. Setting `usage_limit: int = 0` publicly broadcasts zero as the default, allowing infinite-use token creation.

3. Temporal Drift

Using raw `datetime.now()` breaks serialisation consistency. `Timestamp_Now` is mandatory for capturing current time as integer milliseconds.

4. Type Chain Destruction

Explicit `str()` casts physically sever the `Type_Safe` validation chain. The framework handles conversions automatically.

The Pre-Flight Developer Checklist

SCHEMAS

- ☐ Inherits `Type_Safe`
- ☐ All attributes annotated
- ☐ Zero mutable defaults
- ☐ No raw primitives used

COLLECTIONS

- ☐ Prefixed correctly (`Dict__`, `List__`)
- ☐ Pure type definitions
- ☐ Zero methods defined

SERVICES

- ☐ `@type_safe` on public methods
- ☐ Returns `None` (no exceptions)
- ☐ No manual `str()` casts

TESTING

- ☐ `setUpClass` for heavy resources
- ☐ Context manager `_` implemented
- ☐ `.obj()` and `__SKIP__` utilised
- ☐ Inline comments aligned (no docstrings)

Definitive clearance required before submitting Pull Request.